



Pod Security Standards

Pod Security Standards

A detailed look at the different policy levels defined in the Pod Security Standards.

The Pod Security Standards define three different *policies* to broadly cover the security spectrum. These policies are *cumulative* and range from highly-permissive to highly-restrictive. This guide outlines the requirements of each policy.

Profile	Description
Privileged	Unrestricted policy, providing the widest possible level of permissions. This policy allows for known privilege escalations.
Baseline	Minimally restrictive policy which prevents known privilege escalations. Allows the default (minimally specified) Pod configuration.
Restricted	Heavily restricted policy, following current Pod hardening best practices.

Profile Details

Privileged

The *Privileged* policy is purposely-open, and entirely unrestricted. This type of policy is typically aimed at system- and infrastructure-level workloads managed by privileged, trusted users.

The Privileged policy is defined by an absence of restrictions. If you define a Pod where the Privileged security policy applies, the Pod you define is able to bypass typical container isolation mechanisms. For example, you can define a Pod that has access to the node's host network.

Baseline

The *Baseline* policy is aimed at ease of adoption for common containerized workloads while preventing known privilege escalations. This policy is targeted at application operators and developers of non-critical applications. The following listed controls should be enforced/disallowed:

Note:
In this table, wildcards (*) indicate all elements in a list. For example, `spec.containers[*].securityContext` refers to the Security Context object for *all defined containers*. If any of the listed containers fails to meet the requirements, the entire pod will fail validation.

Control	Policy
HostProcess	Windows Pods offer the ability to run HostProcess containers with access to the Windows host machine. Privileged access to the host is disallowed by the Baseline policy.

 **FEATURE STATE:** Kubernetes v1.26 [stable]

Restricted Fields

- `spec.securityContext.windowsOptions.hostProcess`
- `spec.containers[*].securityContext.windowsOptions`
- `spec.initContainers[*].securityContext.windowsOptions`
- `spec.ephemeralContainers[*].securityContext.windowsOptions`

Allowed Values

- Undefined/nil
- `false`

Host Namespaces	Sharing the host namespaces must be disallowed. Restricted Fields • <code>spec.hostNetwork</code> • <code>spec.hostPID</code> • <code>spec.hostIPC</code>
-----------------	---

Allowed Values

- Undefined/nil
- false

Privileged Containers Privileged Pods disable most security mechanisms and must be

Restricted Fields

- `spec.containers[*].securityContext.privileged`
- `spec.initContainers[*].securityContext.privileged`
- `spec.ephemeralContainers[*].securityContext.privi`

Allowed Values

- Undefined/nil
- false

Capabilities

Adding additional capabilities beyond those listed below must k

Restricted Fields

- `spec.containers[*].securityContext.capabilities.a`
- `spec.initContainers[*].securityContext.capabiliti`
- `spec.ephemeralContainers[*].securityContext.capab`

Allowed Values

- Undefined/nil
- AUDIT_WRITE
- CHOWN
- DAC_OVERRIDE
- FOWNER
- FSETID
- KILL
- MKNOD
- NET_BIND_SERVICE
- SETFCAP
- SETGID
- SETPCAP
- SETUID

- `SYS_CHROOT`

HostPath Volumes

HostPath volumes must be forbidden.

Restricted Fields

- `spec.volumes[*].hostPath`

Allowed Values

- Undefined/nil

Host Ports

HostPorts should be disallowed entirely (recommended) or restricted

Restricted Fields

- `spec.containers[*].ports[*].hostPort`
- `spec.initContainers[*].ports[*].hostPort`
- `spec.ephemeralContainers[*].ports[*].hostPort`

Allowed Values

- Undefined/nil
- Known list (not supported by the built-in [Pod Security Admission](#))
- `0`

Host Probes / Lifecycle Hooks (v1.34+)

The Host field in probes and lifecycle hooks must be disallowed

Restricted Fields

- `spec.containers[*].livenessProbe.httpGet.host`
- `spec.containers[*].readinessProbe.httpGet.host`
- `spec.containers[*].startupProbe.httpGet.host`
- `spec.containers[*].livenessProbe.tcpSocket.host`
- `spec.containers[*].readinessProbe.tcpSocket.host`
- `spec.containers[*].startupProbe.tcpSocket.host`
- `spec.containers[*].lifecycle.postStart.tcpSocket.host`
- `spec.containers[*].lifecycle.preStop.tcpSocket.host`
- `spec.containers[*].lifecycle.postStart.httpGet.host`
- `spec.containers[*].lifecycle.preStop.httpGet.host`
- `spec.initContainers[*].livenessProbe.httpGet.host`
- `spec.initContainers[*].readinessProbe.httpGet.host`

- `spec.initContainers[*].startupProbe.httpGet.host`
- `spec.initContainers[*].livenessProbe.tcpSocket.ho`
- `spec.initContainers[*].readinessProbe.tcpSocket.h`
- `spec.initContainers[*].startupProbe.tcpSocket.hos`
- `spec.initContainers[*].lifecycle.postStart.tcpSoc`
- `spec.initContainers[*].lifecycle.preStop.tcpSocke`
- `spec.initContainers[*].lifecycle.postStart.httpGe`
- `spec.initContainers[*].lifecycle.preStop.httpGet.`

Allowed Values

- Undefined/nil
- ""

AppArmor

On supported hosts, the `RuntimeDefault` AppArmor profile is baseline policy should prevent overriding or disabling the default restrict overrides to an allowed set of profiles.

Restricted Fields

- `spec.securityContext.appArmorProfile.type`
- `spec.containers[*].securityContext.appArmorProfil`
- `spec.initContainers[*].securityContext.appArmorPr`
- `spec.ephemeralContainers[*].securityContext.appAr`

Allowed Values

- Undefined/nil
- `RuntimeDefault`
- `Localhost`

-
- `metadata.annotations["container.apparmor.security`

Allowed Values

- Undefined/nil
- `runtime/default`
- `localhost/*`

SELinux

Setting the SELinux type is restricted, and setting a custom SELinux type is forbidden.

Restricted Fields

- `spec.securityContext.seLinuxOptions.type`
- `spec.containers[*].securityContext.seLinuxOptions`
- `spec.initContainers[*].securityContext.seLinuxOpt`
- `spec.ephemeralContainers[*].securityContext.seLin`

Allowed Values

- `Undefined/""`
 - `container_t`
 - `container_init_t`
 - `container_kvm_t`
 - `container_engine_t` (since Kubernetes 1.31)
-

Restricted Fields

- `spec.securityContext.seLinuxOptions.user`
- `spec.containers[*].securityContext.seLinuxOptions`
- `spec.initContainers[*].securityContext.seLinuxOpt`
- `spec.ephemeralContainers[*].securityContext.seLin`
- `spec.securityContext.seLinuxOptions.role`
- `spec.containers[*].securityContext.seLinuxOptions`
- `spec.initContainers[*].securityContext.seLinuxOpt`
- `spec.ephemeralContainers[*].securityContext.seLin`

Allowed Values

- `Undefined/""`
-

`/proc` Mount Type The default `/proc` masks are set up to reduce attack surface, ,

Restricted Fields

- `spec.containers[*].securityContext.procMount`
- `spec.initContainers[*].securityContext.procMount`
- `spec.ephemeralContainers[*].securityContext.procM`

Allowed Values

- Undefined/nil
- Default

Seccomp Seccomp profile must not be explicitly set to `Unconfined` .

Restricted Fields

- `spec.securityContext.seccompProfile.type`
- `spec.containers[*].securityContext.seccompProfile`
- `spec.initContainers[*].securityContext.seccompPro`
- `spec.ephemeralContainers[*].securityContext.secco`

Allowed Values

- Undefined/nil
- RuntimeDefault
- Localhost

Sysctls Sysctls can disable security mechanisms or affect all containers disallowed except for an allowed "safe" subset. A sysctl is consid namespaced in the container or the Pod, and it is isolated from the same Node.

Restricted Fields

- `spec.securityContext.sysctls[*].name`

Allowed Values

- Undefined/nil
- `kernel.shm_rmid_forced`
- `net.ipv4.ip_local_port_range`
- `net.ipv4.ip_unprivileged_port_start`

- `net.ipv4.tcp_syncookies`
- `net.ipv4.ping_group_range`
- `net.ipv4.ip_local_reserved_ports` (since Kubernetes 1.29)
- `net.ipv4.tcp_keepalive_time` (since Kubernetes 1.29)
- `net.ipv4.tcp_fin_timeout` (since Kubernetes 1.29)
- `net.ipv4.tcp_keepalive_intvl` (since Kubernetes 1.29)
- `net.ipv4.tcp_keepalive_probes` (since Kubernetes 1.29)

Restricted

The ***Restricted*** policy is aimed at enforcing current Pod hardening best practices, at the expense of some compatibility. It is targeted at operators and developers of security-critical applications, as well as lower-trust users. The following listed controls should be enforced/disallowed:

Note:

In this table, wildcards (*) indicate all elements in a list. For example, `spec.containers[*].securityContext` refers to the Security Context object for *all defined containers*. If any of the listed containers fails to meet the requirements, the entire pod will fail validation.

Control

Policy

Everything from the Baseline policy

Volume Types

The Restricted policy only permits the following volume types:

Restricted Fields

- `spec.volumes[*]`

Allowed Values

Every item in the `spec.volumes[*]` list must set a non-null value:

- `spec.volumes[*].configMap`
- `spec.volumes[*].csi`
- `spec.volumes[*].downwardAPI`
- `spec.volumes[*].emptyDir`

- `spec.volumes[*].ephemeral`
- `spec.volumes[*].persistentVolumeClaim`
- `spec.volumes[*].projected`
- `spec.volumes[*].secret`

Privilege Escalation (v1.8+)

Privilege escalation (such as via `set-user-ID` or `set-gid`) is not allowed. *This is Linux only policy in v1.25+ (`spec.os.name` is `linux`)*

Restricted Fields

- `spec.containers[*].securityContext.all`
- `spec.initContainers[*].securityContext`
- `spec.ephemeralContainers[*].securityCo`

Allowed Values

- `false`

Running as Non-root

Containers must be required to run as non-root users.

Restricted Fields

- `spec.securityContext.runAsNonRoot`
- `spec.containers[*].securityContext.run`
- `spec.initContainers[*].securityContext`
- `spec.ephemeralContainers[*].securityCo`

Allowed Values

- `true`

The container fields may be undefined/ `nil` if the pod-level `spec.securityContext.runAsNonRoot` is set to `true`.

Running as Non-root user (v1.23+)

Containers must not set `runAsUser` to 0

Restricted Fields

- `spec.securityContext.runAsUser`
- `spec.containers[*].securityContext.run`
- `spec.initContainers[*].securityContext`
- `spec.ephemeralContainers[*].securityCo`

Allowed Values

- any non-zero value
- undefined/null

Seccomp (v1.19+)

Seccomp profile must be explicitly set to one of the `Unconfined` profile and the *absence* of a profile at the *pod* level *policy in v1.25+ (`spec.os.name != windows`)*

Restricted Fields

- `spec.securityContext.seccompProfile.type`
- `spec.containers[*].securityContext.seccompProfile.type`
- `spec.initContainers[*].securityContext.seccompProfile.type`
- `spec.ephemeralContainers[*].securityContext.seccompProfile.type`

Allowed Values

- `RuntimeDefault`
- `Localhost`

The container fields may be undefined/ `nil` if the pod-level `spec.securityContext.seccompProfile.type` field is set. The pod-level field may be undefined/ `nil` if `_all_ container-level`

Capabilities (v1.22+)

Containers must drop `ALL` capabilities, and are only allowed to have the `NET_BIND_SERVICE` capability. *This is Linux only policy (`spec.os.name != windows`)*

Restricted Fields

- `spec.containers[*].securityContext.capabilities`
- `spec.initContainers[*].securityContext.capabilities`
- `spec.ephemeralContainers[*].securityContext.capabilities`

Allowed Values

- Any list of capabilities that includes `ALL`

Restricted Fields

- `spec.containers[*].securityContext.capabilities`
- `spec.initContainers[*].securityContext.capabilities`
- `spec.ephemeralContainers[*].securityContext.capabilities`

Allowed Values

- Undefined/nil
 - NET_BIND_SERVICE
-

Policy Instantiation

Decoupling policy definition from policy instantiation allows for a common understanding and consistent language of policies across clusters, independent of the underlying enforcement mechanism.

As mechanisms mature, they will be defined below on a per-policy basis. The methods of enforcement of individual policies are not defined here.

Pod Security Admission Controller

- Privileged namespace
- Baseline namespace
- Restricted namespace

Alternatives

Note: This section links to third party projects that provide functionality required by Kubernetes. The Kubernetes project authors aren't responsible for these projects, which are listed alphabetically. To add a project to this list, read the [content guide](#) before submitting a change. [More information](#).

Other alternatives for enforcing policies are being developed in the Kubernetes ecosystem, such as:

- [Kubewarden](#)
- [Kyverno](#)
- [OPA Gatekeeper](#)

Pod OS field

Kubernetes lets you use nodes that run either Linux or Windows. You can mix both kinds of node in one cluster. Windows in Kubernetes has some limitations and

differentiators from Linux-based workloads. Specifically, many of the Pod `securityContext` fields [have no effect on Windows](#).

Note:

Kubelets prior to v1.24 don't enforce the pod OS field, and if a cluster has nodes on versions earlier than v1.24 the Restricted policies should be pinned to a version prior to v1.25.

Restricted Pod Security Standard changes

Another important change, made in Kubernetes v1.25 is that the *Restricted* policy has been updated to use the `pod.spec.os.name` field. Based on the OS name, certain policies that are specific to a particular OS can be relaxed for the other OS.

OS-specific policy controls

Restrictions on the following controls are only required if `.spec.os.name` is not `windows` :

- Privilege Escalation
- Seccomp
- Linux Capabilities

User namespaces

User Namespaces are a Linux-only feature to run workloads with increased isolation. How they work together with Pod Security Standards is described in the [documentation](#) for Pods that use user namespaces.

FAQ

Why isn't there a profile between Privileged and Baseline?

The three profiles defined here have a clear linear progression from most secure (Restricted) to least secure (Privileged), and cover a broad set of workloads. Privileges required above the Baseline policy are typically very application specific, so we do not offer a standard profile in this niche. This is not to say that the privileged profile should always be used in this case, but that policies in this space need to be defined on a case-by-case basis.

SIG Auth may reconsider this position in the future, should a clear need for other profiles arise.

What's the difference between a security profile and a security context?

[Security Contexts](#) configure Pods and Containers at runtime. Security contexts are defined as part of the Pod and container specifications in the Pod manifest, and represent parameters to the container runtime.

Security profiles are control plane mechanisms to enforce specific settings in the Security Context, as well as other related parameters outside the Security Context. As of July 2021, [Pod Security Policies](#) are deprecated in favor of the built-in [Pod Security Admission Controller](#).

What about sandboxed Pods?

There is currently no API standard that controls whether a Pod is considered sandboxed or not. Sandbox Pods may be identified by the use of a sandboxed runtime (such as gVisor or Kata Containers), but there is no standard definition of what a sandboxed runtime is.

The protections necessary for sandboxed workloads can differ from others. For example, the need to restrict privileged permissions is lessened when the workload is isolated from the underlying kernel. This allows for workloads requiring heightened permissions to still be isolated.

Additionally, the protection of sandboxed workloads is highly dependent on the method of sandboxing. As such, no single recommended profile is recommended for all sandboxed workloads.

Feedback

Was this page helpful?

Yes

No

Last modified August 06, 2025 at 6:48 PM PST: [nit-fix: Add empty value for host field in probes PSA \(a0fb9cc6b3\)](#)

Items on this page refer to third party products or projects that provide functionality required by Kubernetes. The Kubernetes project authors aren't responsible for those third-party products or projects. See the [CNCF website guidelines](#) for more details.

You should read the [content guide](#) before proposing a change that adds an extra third-party link.